Implementation of a Metaheuristic for the Steiner Tree Problem in Graphs

Martin Josef Geiger*

April 30, 2018

1 Introduction

This is our contribution to Track 3 of the *Parameterized Algorithms and Computational Experiments Challenge* PACE 2018. The submitted source-code is a heuristic algorithm with the goal of finding best-possible Steiner trees in graphs.

In relation to the taxonomy of metaheuristics, the approach is best described as an *Iterated Local Search* algorithm [5] with some sort of *noising* applied on the data.

2 Components

Our approach brings together the following components:

Preprocessing/ reduction of the instances is supported. Nodes/ edges are removed based on cardinality checks, pruning Steiner nodes with one or two edges. Besides, edges that can be replaced by a not longer path are identified, also. Moreover, edges close to terminals are searched and eliminated.

The preprocessing is run in an incomplete manner, simply because, for an arbitrary input, not all shortest paths can be computed within the given time limit (which is set to 18 seconds for the entire preprocessing).

Our work is primarily based on [3]. There are several other publications on reduction approaches, but we foremost relied on this classical

*m.j.geiger@hsu-hh.de

publication. The best-possible preprocessing of graphs for the Steiner tree problem under running time restrictions is not obvious, as we have learned from [1]. Ultimately, this has resulted in the chosen running time cap.

The Voronoi diagram computation lies the foundation for the construction of some first feasible solutions: the Voronoi diagram delivers information on terminals that are neighbors (i. e., are bases of neighboring Voronoi regions).

Voronoi diagrams for the Steiner tree problem in graphs are discussed in several publications. Our work is primarily inspired by [6, 8].

Greedy construction of feasible solutions, in which terminals are linked to a tree, one by one, until all terminals are connected. This procedure returns a subgraph for the given problem, for which there exists a connecting spanning tree. This subgraph always includes all terminals, and typically includes some additional Steiner nodes, chosen when linking a terminal to the subsequently defined subgraph.

Of course, greedy construction procedures such as the one implemented here can be traced back several years. We foremost acknowledge the work of [7, 9], who, from our perspective, provide instructive explanations on this.

Krukals algorithm [4] for computing a minimum spanning tree, including a subsequent pruning step for removing unnecessary Steiner nodes and their connecting edges.

A randomized deconstruction and reconstruction of a current solution, which includes a randomization of the edge values. In this procedure, a Steiner node with at least three connecting edges (alternatively: a terminal) is removed, and a current tree is split into unconnected trees. Hence, when running this move, a current tree is split into at least three subtrees. In combination with the randomization of the edge values, this component plays the role of a *diversification* of solutions during search.

The randomization of the edge values has been implemented as described in [7]. Several other variants have been tested, also. Based on our experiments, we believe that the work of [7] presents truly a very well-chosen and well-parameterized approach.

A split-and-reconnect move, splitting a current tree into exactly two subtrees and reconnecting them optimally (i. e., cost-minimal). In contrast

to the deconstruction mentioned before, this component is more of an *intensification* procedure.

A variant of Dijkstras algorithm [2] for connecting a tree with another tree by means of a shortest path. This shortest path algorithm comes with the twist of randomizing the selection step for the next edge/ node, which allows the identification of structurally different paths in case there are multiple, equally long, shortest paths.

To the best of our knowledge, we have not seen such a variant of Dijkstras algorithm in the literature, although it can be suspected that something similar has been done before.

A simple caching of paths which have been chosen for the split-and-reconnect, based on a hash table. Note that not the actual path is cached, but only the information from which node to which other node it has been going. Therefore, the caching only occupies little memory.

A simple, adaptive control mechanism that keeps track of the algorithm and re-starts the search in case the current solution cannot be improved within a given time limit. "Adaptive" means that the time limit is dynamically adapted depending on the progress of the algorithm.

An attempt has been made to come up with a short, straight-forward implementation: the Microsoft Visual Studio code editor counts **761 lines of code**.

The scientific value of having a short code is certainly debatable. However, such an attempt has been made, specifically as we are proposing a heuristic algorithm. Heuristics are commonly considered to be "easy", "generic" procedures. Unfortunately, there are many scientific contributions of heuristics that are not easy at all: they come with many control parameters, complex/ lengthy implementations, etc. It was our intention to *not go* in this direction. Quite to the contrary, we wanted to show how easy a (nevertheless hopefully effective) approach can be.

Ultimately, this implies that each element of our concept has been tested for a positive contribution towards solving the problem better. We kept what, in our experience, works.

3 Remarks

1. The algorithm has been coded in Microsoft Visual Basic under Windows (v10.0). However, as the target platform of the competition,

`optil.io` [10, 11], runs Linux, we tried to enure that the program also compiles and runs there. This was possible with the very kind support provided by the administrators of `optil.io`, who added the Visual Basic support under their Linux platform upon our request.

Numerous runs have been carried out on our computer and the target platform. On this basis, we believe that the implementation is quite stable and reliable.

2. To the best of our knowledge, Visual Basic cannot cope with the SIGTERM signal. However, we made sure that the running time does not exceed the given 1800 seconds (running time is measured as wall-clock-time, not CPU-time or other). This number is a fixed parameter in the source code (feel free to modify it and test other running times). Extensive experiments have been carried out to ensure that the program does indeed terminate after the given running time.

   When manually altering the running time, it is necessary to choose a value greater than the maximum running time for the preprocessing phase (which is 18 seconds).

3. The program is submitted to Track 3 of the competition. It also has been tested on the instances of Tracks 1 and 2. In fact, the algorithm is able to find quite some optimal solutions. However, and due to the nature of the approach, it cannot prove optimality, so obviously, we do not submit the program to any other track than Track 3.

Acknowledgements

We would like to thank two groups of people. On the one hand, we acknowledge the work of the PACE 2018 Challenge organizers, Florian Sikora and Edouard Bonnet, who set up this difficult competition. On the other hand, we thank the head and team of `optil.io`, Szymon Wasik and colleagues, for providing and maintaining such a useful optimization platform [10, 11].

References

[1] Stephan Beyer and Markus Chimani. The influence of preprocessing on Steiner tree approximations. In Zaixin Lu, Donghyun Kim, Weili Wu, Wei Li, and Ding-Zhu Du, editors, *Combinatorial Optimization and Applications*, volume 9486 of *Lecture Notes in Computer Science*, pages 601–616. Springer, Cham, 2015.

[2] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, pages 269–271, 1959.

[3] C. W. Duin and A. Volgenant. Reduction tests for the Steiner problem in graphs. *Networks*, 19:549–567, 1989.

[4] Joseph Kruskal. On the shortest spanning subtree and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7:48–50, 1956.

[5] Helena R. Lourenço, Olivier Martin, and Thomas Stützle. Iterated local search. In Fred Glover and Gary A. Kochenberger, editors, *Handbook of Metaheuristics*, volume 57 of *International Series in Operations Research & Management Science*, chapter 11, pages 321–353. Kluwer Academic Publishers, Boston, Dordrecht, London, 2003.

[6] Kurt Mehlhorn. A faster approximation algorithm for the Steiner problem in graphs. *Information Processing Letters*, 27:125–128, 1988.

[7] Thomas Pajor, Eduardo Uchoa, and Renato F. Werneck. A robust and scalable algorithm for the Steiner problem in graphs. *Mathematical Programming Computation*, 10(1):69–118, 2018.

[8] Eduardo Uchoa and Renato F. Werneck. Fast local search for the Steiner problem in graphs. *Journal of Experimental Algorithmics*, 17:2.2, 2012.

[9] Stefan Voß. Steiner's problem in graphs: heuristic methods. *Discrete Applied Mathematics*, 40:45–72, 1992.

[10] Szymon Wasik, Maciej Antczak, Jan Badura, Artur Laskowski, and Tomasz Sternal. Optil.io: Cloud based platform for solving optimization problems using crowdsourcing approach. In *CSCW '16 Companion – Proceedings of the 19th ACM Conference on Computer Supported Cooperative Work and Social Computing Companion*, pages 433–436, San Francisco, California, USA, February 26–March 2, 2016.

[11] Szymon Wasik, Maciej Antczak, Jan Badura, Artur Laskowski, and Tomasz Sternal. A survey on online judge systems and their applications. *ACM Computing Surveys (CSUR)*, 51(1):article No. 3, 2018.